

Online Text-to-Speech (TTS)

Online Text-to-Speech (TTS)

1. Concept Introduction
 - 1.1 What is "TTS"?
 - 1.2 Implementation Principles Overview
 1. **Text Analysis**
 2. **Language Processing**
 3. **Speech Synthesis**
 4. **Sound Waveform Generation**
2. Code Analysis
 - Key Code
 1. TTS Initialization and Invocation (`largemodel/largemodel/model_service.py`)
 2. TTS Backend Implementation (`largemodel/utis/large_model_interface.py`)
 - Code Analysis
3. Practical Operations
 - 3.1 Configure Online TTS
 - 3.2 Start and Test the Function

1. Concept Introduction

1.1 What is "TTS"?

TTS technology is a technology that converts written text into audible speech output for humans. It enables computers to "read aloud" text content and is widely used in accessibility reading, intelligent assistants, navigation systems, educational software, and many other fields. With TTS, users can hear naturally flowing human voices generated by machines, greatly improving the convenience and flexibility of information access.

1.2 Implementation Principles Overview

The implementation of TTS systems mainly includes the following key steps and technologies:

1. Text Analysis

- In this stage, the input text is first preprocessed, including but not limited to removing irrelevant characters, standardizing punctuation marks and capitalization, word segmentation, and identifying numbers and other special symbols and converting them into corresponding word forms.
- At the same time, linguistic analysis of the text is also required, such as determining the pronunciation of each word (which usually requires using a pronunciation dictionary), stress positions, intonation patterns, and sentence structure.

2. Language Processing

- This step mainly focuses on how to pronounce correctly and adjust intonation based on context. For example, the word "read" has different pronunciations in different tenses (past/past participle pronounced as /red/, while in other cases pronounced as /ri:d/). Therefore, a powerful language model is needed to understand these subtle differences.
- Prosody modeling is also involved here, which decides which parts should be emphasized, whether the speech rate should be fast or slow, and the emotional color of the entire

sentence.

3. Speech Synthesis

- After the text information processed in the previous two stages is sent to the speech synthesis engine, which is responsible for generating the actual sound waveform.
- Traditional TTS systems use concatenative synthesis methods, i.e., selecting appropriate units from a database of pre-recorded speech fragments to concatenate to form complete sentences. Although this method can produce high-quality speech, it is limited by the samples in the database.
- Modern TTS systems rely more on parametric synthesis or neural network synthesis (such as WaveNet, Tacotron, etc.). These methods can directly predict speech features from text and generate continuous sound signals. In particular, deep learning-based methods can better capture subtle variations in speech, producing more natural and smooth sounds.

4. Sound Waveform Generation

- Finally, the generated sound waveform is further processed to ensure its quality meets the expected standards, such as adjusting volume and equalizing frequency response.
- After that, these audio data can be played through speakers or other audio playback devices for people to listen to.

With the development of artificial intelligence and machine learning technologies, especially the application of deep learning, TTS systems have not only improved significantly in accuracy but also made great progress in naturalness and emotional expression, making machine-generated speech increasingly close to real human voices.

2. Code Analysis

Key Code

1. TTS Initialization and Invocation

(`largemodel/largemodel/model_service.py`)

```
# From largemodel/largemodel/model_service.py
class LargeModelService(Node):
    def __init__(self):
        # ...
        self.init_param_config()
        # ...

    def system_sound_init(self):
        """Initialize TTS system"""
        model_type = "online" if self.use_onlinetts else "local"
        self.model_client.tts_model_init(model_type, self.language)
        self.get_logger().info(f'TTS initialized with {model_type} model')

    def play_tts_audio(self):
        """Play TTS audio file"""
        try:
            if not os.path.exists(self.tts_out_path):
                self.get_logger().error(f"Audio file not found: {self.tts_out_path}")
            return

            # Use aplay command to play audio
```

```

        self.get_logger().info(f"🔊 Playing audio: aplay -D plughw:0,0 {self.tts_out_path}")

        # Blocking playback
        result = subprocess.run(
            ["mpg123", "-a", "plughw:0,0", self.tts_out_path],
            capture_output=True,
            text=False
        )

        if result.returncode == 0:
            self.get_logger().info("TTS playback completed successfully")
        else:
            self.get_logger().error(f"TTS playback failed: {result.stderr}")

    except Exception as e:
        self.get_logger().error(f"Error playing TTS audio: {e}")

```

2. TTS Backend Implementation

(`largemodel/utis/large_model_interface.py`)

```

# From largemodel/utis/large_model_interface.py
class model_interface:
    # ...
    def tts_model_init(self, model_type='oline', language='zh'):
        if model_type=='oline':
            if self.tts_supplier=='baidu':
                self.token=self.fetch_token()

                self.model_type='oline'
            elif model_type=='local':
                self.model_type='local'
            if language=='zh':
                tts_model=self.zh_tts_model
                tts_json=self.zh_tts_json
            elif language=='en':
                tts_model=self.en_tts_model
                tts_json=self.en_tts_json
            self.synthesizer = piper.PiperVoice.load(tts_model,
                config_path=tts_json, use_cuda=False)

```

Code Analysis

The Text-to-Speech (TTS) function is initiated by the `LargeModelService` node and provided by the `model_interface` class. Its design switches between different backend services through parameter configuration.

1. Initialization Flow (`model_service.py`):

- When `LargeModelService` is initialized, the `init_param_config` function reads the boolean value `useolinetts` from the ROS parameter server.
- The `system_sound_init` function determines whether to pass `'local'` or `'oline'` String to the `self.model_client.tts_model_init` method based on the value of `useolinetts`.

- In `large_model_interface.py`, the `tts_model_init` method executes the corresponding initialization logic based on the received String parameter. If `'local'`, it uses `piper.PiperVoice.load` to load the local model file.

2. Synthesis and Playback Flow (`model_service.py`):

- When speech playback is needed, the `_safe_play_audio` function is called.
- This function first calls the `self.model_client.voice_synthesis` method, passing in the text to be converted and the target audio path `self.tts_out_path`.
- After the `voice_synthesis` method finishes executing and generates the audio file, `_safe_play_audio` then calls `self.play_audio_async` to asynchronously play the file.

3. Backend Implementation Selection (`large_model_interface.py`):

- The `voice_synthesis` method is the backend distribution center for TTS functionality. Internally, it selects the execution path by checking the `self.model_type` property value set during initialization.
- If `self.model_type` is `'local'`, the code block uses Python's `wave` library to open a WAV file, sets its header parameters (channels, sample width, sample rate), and then calls the `self.synthesizer.synthesize` method to directly write the audio stream synthesized from text into the file.
- If `self.model_type` is `'oline'`, it enters the logic branch for different cloud service providers (such as Baidu, Aliyun).
- This architecture separates the upper-layer node's call ("say this paragraph") from the lower-layer specific synthesis technology (which library to use, which API to call).

3. Practical Operations

3.1 Configure Online TTS

1. Open the model interface configuration file `large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

2. Fill in your API credentials:

Find the online TTS section. Here taking Baidu as an example, fill in your Key, and you can also specify a voice name you like.

```
# large_model_interface.yaml

# Baidu Cloud TTS Configuration
baidu_API_KEY: 'xxxxxxxxxxxxxxxxxxxxxxxx' # Baidu platform API key
baidu_SECRET_KEY: 'xxxxxxxxxxxxxxxxxxxxxxxx' # Baidu platform SECRET key
CUID: 'IN7dAbz1thDJKVhLdypkpB6sDVG86xeG' # Device identifier
PER: 4 # Voice: Basic voice library (0=Dumei, 1=Duxiaoyu, 3=Duxiaoyao, 4=Duyaya)
SPD: 5 # Speech rate (0-15), default 5
PIT: 5 # Pitch (0-15), default 5
VOL: 5 # volume (0-9), default 5
```

3. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

4. Enable online TTS mode:

Set the `useonline tts` parameter to `True`.

```
# yahboom.yaml

tts_only_node:                                # TTS only node parameters
  ros__parameters:

    useonline tts: True                        # whether to use online TTS
```

3.2 Start and Test the Function

1. Open a terminal and start the TTS node:

Run the following command:

```
ros2 launch largemodel tts_only.launch.py
```

```
[asr-2] 2026-04-03 15:25:37.015110135 [W:onnxruntime:Default, device_discovery.cc:164 DiscoverDevicesForPlatform] GPU device discovery failed: device_discovery.cc:89 ReadFileContents Failed to open file: "/sys/class/drm/card0/device/vendor"
[model_service-1] [INFO] [1775201187.190192162] [model_service]: 🚀 Initializing LargeModelService node...
[model_service-1] [INFO] [1775201187.200708316] [model_service]: ✅ Found package at: /home/sunrise/yahboomcar_ws/src/largemodel
[model_service-1] [INFO] [1775201187.222228913] [model_service]: 📄 Parameters loaded: language-zh, platform=tongyi
[model_service-1] [INFO] [1775201187.225283397] [model_service]: 🚀 Initializing ROS2 communication...
[model_service-1] [INFO] [1775201187.264556487] [model_service]: ✅ Subscribed to /prompt_text topic
[model_service-1] [INFO] [1775201187.279347577] [model_service]: ✅ Subscribed to /tts_topic
[model_service-1] [INFO] [1775201187.366040089] [model_service]: ✅ ROS2 communication initialized
[model_service-1] [INFO] [1775201187.376645742] [model_service]: Initializing ToolsManager...
[model_service-1] [INFO] [1775201187.384539535] [model_service]: Initializing AIAgent...
[model_service-1] [INFO] [1775201187.887850191] [model_service]: AI Agent initialized successfully
[model_service-1] [INFO] [1775201187.900870791] [model_service]: Initializing MCP Server...
[model_service-1] [INFO] [1775201187.976679650] [model_service]: Registered 6 tools from Tool Chain Manager
[model_service-1] [INFO] [1775201187.983477074] [model_service]: MCP Server initialized with Tool Chain Manager
[model_service-1] [INFO] [1775201187.989562751] [model_service]: 🚀 Initializing LLM platform: tongyi
[model_service-1] Tongyi (Dashscope) client initialized successfully. Using model qwen-vl-max
[asr-2] [INFO] [1775201188.660028797] [asr]: ASR Node initialized successfully. Waiting for audio io...
[model_service-1] [INFO] [1775201188.747261639] [model_service]: 🚀 TTS initialized with online model
[model_service-1] [INFO] [1775201188.760332780] [model_service]: ✅ LLM platform tongyi initialized successfully
[model_service-1] [INFO] [1775201188.922985609] [model_service]: 📄 消息文件初始化: /home/sunrise/yahboomcar_ws/src/largemodel/resources_file/conversation_message.json
```

2. Send text to be synthesized:

Open a new terminal and run the following command to publish a voice broadcast message:

```
ros2 topic pub --once /tts_topic std_msgs/msg/String '{data: "TTS test successful"}'
```

3. Test:

If everything is normal, you should hear the robot speak "TTS test successful" with synthesized voice from your speaker.